EMO Robot

Claude Code Agent — Product Requirements Document

Panduan coding lengkap untuk Claude Code Agent | v1.0

7 Modul	1 Folder	Python 3.11	RPi Zero 2W
package struktur	emorobot/	runtime utama	target platform

Dokumen	PRD Claude Code Agent — EMO Robot
Versi	1.0
Tanggal	29 Juni 2026
Author	Dimas
Target	Claude Code CLI / Claude Code Desktop
Platform	Raspberry Pi Zero 2W + Python 3.11
Root folder	~/emorobot/
Status	Ready to implement — Phase 1 aktif

1. Overview & Tujuan

Dokumen ini adalah PRD (Product Requirements Document) yang ditujukan untuk Claude Code Agent — bukan manusia. Claude Code akan membaca dokumen ini dan secara otomatis membuat seluruh struktur folder, file Python, konfigurasi, dan skrip yang dibutuhkan untuk menjalankan Mini Robot EMO-style pada Raspberry Pi Zero 2W.

Cara Pakai dengan Claude Code

```
# Install Claude Code
npm install -g @anthropic-ai/claude-code
# Jalankan di folder project
cd ~/emorobot
claude
# Lalu paste instruksi:
# "Baca PRD ini dan buat seluruh struktur folder dan file sesuai spesifikasi"
```

Prinsip Arsitektur

- Semua modul bersifat independen — bisa dijalankan/ditest sendiri
- Komunikasi antar modul via event queue (threading.Queue) — tidak ada import silang
- Setiap modul punya __init__.py, config default, dan unit test minimal
- Main entry point: main.py di root folder
- Konfigurasi terpusat di config.py — tidak ada hardcode di modul
- Logging terpusat via modul nyawa/logger.py
- Autostart via systemd service — robot langsung jalan saat boot

2. Struktur Folder Lengkap

```
~/emorobot/ # ROOT PROJECT
├── main.py # Entry point utama
├── config.py # Semua konfigurasi terpusat
├── requirements.txt # Python dependencies
├── setup.sh # Script install otomatis
├── emorobot.service # Systemd autostart service
├── README.md # Dokumentasi project
│
├── wajah/ # Modul display & ekspresi
│ ├── __init__.py
│ ├── display.py # Driver ST7789 1.3"
│ ├── faces.py # Semua fungsi render ekspresi
│ ├── animator.py # Engine animasi frame-by-frame
│ ├── assets/ # Sprite & font
│ ├── fonts/
│ └── sprites/
│
├── jiwa/ # State machine & mood engine
│ ├── __init__.py
│ ├── mood.py # MoodEngine + state machine
│ ├── personality.py # Karakter & nama robot
│ └── events.py # Definisi semua event type
│
├── telinga/ # Input: mic, touch, ultrasonik
│ ├── __init__.py
│ ├── microphone.py # USB mic input + clap detect
│ ├── touch.py # Touch sensor TTP223
│ ├── proximity.py # Ultrasonik HC-SR04
│ └── voice_trigger.py # Wake word detection
│
├── mulut/ # Output: audio, TTS, speaker
│ ├── __init__.py
│ ├── speaker.py # Playback audio via PAM8406
│ ├── sounds.py # Sound effect manager
│ ├── tts.py # Text-to-speech (espeak-ng)
│ ├── assets/sounds/ # File .wav reaksi robot
│ ├── startup.wav
│ ├── happy.wav
│ ├── sad.wav
│ ├── curious.wav
│ ├── excited.wav
│ ├── sleepy.wav
│ └── touched.wav
│
├── mata/ # Kamera + face detection (Phase 2)
│ ├── __init__.py
│ ├── camera.py # Pi Camera init + capture
│ ├── detector.py # OpenCV face detection
│ └── recognizer.py # Face recognition (dlib)
│
├── otak/ # AI: ChatGPT + memory (Phase 3)
│ ├── __init__.py
│ ├── chat.py # OpenAI API integration
│ ├── memory.py # Conversation memory
│ └── education.py # Mode edukasi anak
│
├── nyawa/ # Core: power, logging, system
├── __init__.py
├── logger.py # Centralized logging
├── power.py # Battery monitoring
├── system.py # GPIO init, shutdown handler
└── event_bus.py # Global event queue
```

3. Spesifikasi Per Modul

WAJAH

Display ST7789 1.3" + Engine Animasi Ekspresi

Tanggung Jawab

- Inisialisasi dan drive layar ST7789 1.3" 240×240 via SPI
- Render semua ekspresi wajah (happy, sad, curious, sleepy, excited, surprised, bored)
- Animasi frame-by-frame: mata berkedip, mulut bergerak saat bicara, transisi antar mood
- Overlay teks nama saat face recognition aktif (Phase 2)
- Battery indicator di pojok layar
- Idle screensaver saat tidak ada aktivitas > 5 menit

File: wajah/display.py

```
class DisplayDriver:
def __init__(self, config: DisplayConfig)
def init() -> None # SPI init + backlight on
def show(image: PIL.Image) -> None # Render PIL image ke layar
def brightness(level: int) -> None # 0-100%
def off() -> None # Matikan backlight
def on() -> None
```

File: wajah/faces.py

```
# Semua fungsi return PIL.Image (240x240 RGB)
def face_happy(tick: int, blink: bool) -> Image
def face_sad(tick: int) -> Image
def face_curious(tick: int) -> Image
def face_sleepy(tick: int) -> Image
def face_excited(tick: int) -> Image
def face_surprised(tick: int) -> Image
def face_bored(tick: int) -> Image
def face_talking(tick: int, intensity: float) -> Image # mulut animasi
def face_loading() -> Image # boot screen
```

File: wajah/animator.py

```
class Animator:
def __init__(self, display: DisplayDriver, mood_engine: MoodEngine)
def start() -> None # Start animation loop thread (20 FPS)
def stop() -> None
def set_mood(mood: str) -> None
def trigger_blink() -> None
def set_talking(active: bool, intensity: float) -> None
def show_name(name: str, duration: float) -> None
def show_battery(level: int) -> None
```

Konfigurasi (config.py)

```
DisplayConfig:
SPI_PORT = 0
SPI_CS = 0 # GPIO8 (Pin 24)
DC_PIN = 24 # GPIO24 (Pin 18)
RST_PIN = 25 # GPIO25 (Pin 22)
BL_PIN = 18 # GPIO18 (Pin 12)
WIDTH = 240
HEIGHT = 240
SPI_SPEED_HZ = 80_000_000
ROTATION = 90
FPS = 20
BLINK_INTERVAL_MIN = 3.0 # detik
BLINK_INTERVAL_MAX = 7.0
BG_COLOR = (10, 10, 20)
```

JIWA

State Machine Mood + Kepribadian Robot

Tanggung Jawab

- Mengelola state mood robot: idle, happy, sad, curious, sleepy, excited, surprised, bored
- Probabilistic transition — setiap mood punya durasi dan probabilitas pindah ke mood lain
- Menerima event dari semua modul dan mengubah mood sesuai event
- Menyimpan kepribadian robot: nama, karakter, preferred responses
- Mood history untuk konteks percakapan AI (Phase 3)

File: jiwa/mood.py

```
MOODS = ['idle','happy','sad','curious','sleepy','excited','surprised','bored']
TRANSITIONS = {
'idle': [('happy',0.3),('curious',0.3),('sleepy',0.2),('bored',0.2)],
'happy': [('idle',0.4),('excited',0.3),('curious',0.3)],
'sad': [('idle',0.5),('curious',0.3),('happy',0.2)],
'curious': [('idle',0.4),('happy',0.3),('excited',0.3)],
'sleepy': [('idle',0.6),('sad',0.2),('sleepy',0.2)],
'excited': [('happy',0.4),('idle',0.3),('curious',0.3)],
'surprised':[('curious',0.4),('happy',0.3),('idle',0.3)],
'bored': [('idle',0.5),('curious',0.3),('sleepy',0.2)],
}
class MoodEngine:
def __init__(self, event_bus: EventBus)
def update() -> None # Panggil tiap tick
def trigger(event: str) -> None # External event trigger
@property mood -> str
@property tick -> int
```

File: jiwa/events.py — Semua Event yang Dikenali

Event	Trigger dari	Efek mood
FACE_DETECTED	mata/detector.py	→ happy
FACE_LOST	mata/detector.py	→ sad / bored
FACE_RECOGNIZED	mata/recognizer.py	→ happy + show name
TOUCHED_HEAD	telinga/touch.py	→ excited
TOUCHED_BODY	telinga/touch.py	→ curious
OBJECT_CLOSE	telinga/proximity.py	→ surprised
CLAP_DETECTED	telinga/microphone.py	→ surprised

VOICE_COMMAND	telinga/voice_trigger.py	→ curious + listen
SPEAKING_START	mulut/tts.py	→ talking animation on
SPEAKING_END	mulut/tts.py	→ talking animation off
BATTERY_LOW	nyawa/power.py	→ sleepy
SHUTDOWN	nyawa/system.py	→ sad animation + bye sound

TELINGA

Input: Mikrofon + Touch Sensor + Ultrasonik

Tanggung Jawab

- Capture audio dari USB microphone — streaming mode
- Deteksi clap (suara keras tiba-tiba) via amplitude threshold
- Wake word detection: 'Hei Emo' (via snowboy atau vosk lokal)
- Baca touch sensor TTP223 via GPIO — interrupt-based (bukan polling)
- Baca sensor ultrasonik HC-SR04 — cek jarak setiap 200ms
- Kirim semua event ke EventBus — tidak ada logika mood di sini

File: telinga/microphone.py

```
class MicrophoneListener:
def __init__(self, event_bus: EventBus, config: MicConfig)
def start() -> None # Start listener thread
def stop() -> None
def record_clip(duration: float) -> bytes # Capture audio untuk STT
def _detect_clap(chunk: bytes) -> bool # Internal: amplitude check
```

File: telinga/touch.py

```
class TouchSensor:
HEAD_PIN = 17 # GPIO17 (Pin 11) — touch sensor 1
BODY_PIN = 27 # GPIO27 (Pin 13) — touch sensor 2
def __init__(self, event_bus: EventBus)
def start() -> None # GPIO interrupt setup + start listener
def stop() -> None
```

Konfigurasi

```
MicConfig:
DEVICE_INDEX = None # Auto-detect USB mic
SAMPLE_RATE = 44100
CHUNK_SIZE = 1024
CHANNELS = 1 # Mono
CLAP_THRESHOLD = 3000 # Amplitude threshold
PROXIMITY_CM = 15 # Jarak 'terlalu dekat' (cm)
TOUCH_HEAD_PIN = 17 # GPIO BCM
TOUCH_BODY_PIN = 27
TRIG_PIN = 23 # HC-SR04 TRIG
ECHO_PIN = 24 # HC-SR04 ECHO — ■■ pakai voltage divider!
```

MULUT

Output: Speaker + Sound Effects + TTS

Tanggung Jawab

- Play sound effect .wav via pygame.mixer — non-blocking (thread)

- Text-to-speech via espeak-ng (offline) — kirim event SPEAKING_START/END
- Queue system: saat robot sedang bicara, sound berikutnya diqueue
- Volume control via software (pygame mixer volume 0.0–1.0)
- Generate sound effect startup saat boot pertama
- Mute saat battery critical

File: mulut/speaker.py

```
class Speaker:
def __init__(self, event_bus: EventBus, config: AudioConfig)
def init() -> None # pygame.mixer.init()
def play(sound_id: str) -> None # Non-blocking play
def play_file(path: str) -> None # Play arbitrary .wav
def stop() -> None
def set_volume(level: float) -> None # 0.0 - 1.0
@property is_playing -> bool
```

File: mulut/tts.py

```
class TextToSpeech:
def __init__(self, event_bus: EventBus, config: AudioConfig)
def speak(text: str, lang: str = 'id') -> None # Fire & forget
def speak_sync(text: str) -> None # Blocking
def stop() -> None
# Internal: subprocess espeak-ng
# Emit SPEAKING_START sebelum bicara
# Emit SPEAKING_END setelah selesai
# espeak-ng -v id+m3 -s 140 -p 60 '{text}'
```

Sound Effect IDs yang Wajib Ada

Sound ID	File	Trigger Event	Durasi
startup	startup.wav	Boot selesai	~1.5 detik
happy	happy.wav	FACE_DETECTED, mood=happy	~0.8 detik
sad	sad.wav	FACE_LOST	~1.0 detik
curious	curious.wav	mood=curious	~0.6 detik
excited	excited.wav	TOUCHED_HEAD	~0.7 detik
sleepy	sleepy.wav	mood=sleepy	~1.2 detik
touched	touched.wav	TOUCHED_BODY	~0.5 detik
surprised	surprised.wav	OBJECT_CLOSE, CLAP_DETECTED	~0.4 detik
shutdown	shutdown.wav	SHUTDOWN event	~1.0 detik
greeting	greeting.wav	FACE_RECOGNIZED	~1.0 detik

Catatan: Jika file .wav belum tersedia, generate placeholder menggunakan espeak-ng atau download dari freesound.org (lisensi CC0). Claude Code wajib generate placeholder saat setup.

MATA

Kamera + Face Detection + Recognition (Phase 2)

Status: Phase 2 — Belum diimplementasi di Phase 1

Modul ini dibuat strukturnya sekarang (stub), tapi implementasi penuh di Phase 2. Claude Code wajib membuat file dengan class dan method stub (pass) agar import tidak error.

File: mata/camera.py — Stub Phase 1

```
class Camera: # Phase 2
def __init__(self, config: CameraConfig): pass
def start() -> None: pass
def stop() -> None: pass
def capture() -> Optional[np.ndarray]: return None
class FaceDetector: # Phase 2
def __init__(self, camera: Camera, event_bus: EventBus): pass
def start() -> None: pass
def stop() -> None: pass
class FaceRecognizer: # Phase 2
def train(name: str, images: List) -> None: pass
def identify(face_img) -> Optional[str]: return None
```

OTAK

ChatGPT AI + Memory + Mode Edukasi (Phase 3)

Status: Phase 3 — Stub only

```
class ChatEngine: # Phase 3
def __init__(self, config: ChatConfig): pass
def chat(prompt: str, context: dict) -> str: return ''
def clear_memory() -> None: pass
class EducationMode: # Phase 3
def quiz_numbers() -> str: return ''
def quiz_alphabet() -> str: return ''
def story_mode() -> str: return ''
```

NYAWA

Core System: Power + Logging + Event Bus

Tanggung Jawab

- EventBus: threading.Queue global untuk komunikasi antar modul
- Logger: logging terpusat ke file /var/log/emorobot.log + console
- Power: monitor baterai via GPIO ADC atau baca voltage dari power module
- System: inisialisasi GPIO, signal handler (SIGTERM, SIGINT), graceful shutdown
- Watchdog: restart modul yang crash otomatis

File: nyawa/event_bus.py

```
from queue import Queue
from dataclasses import dataclass
from typing import Any, Optional
@dataclass
class Event:
type: str # Contoh: 'FACE_DETECTED', 'TOUCHED_HEAD'
data: Any = None # Payload opsional
source: str = '' # Modul pengirim
class EventBus:
_queue: Queue = Queue(maxsize=100)
def publish(event: Event) -> None
def subscribe() -> Event # Blocking get
def subscribe_nowait() -> Optional[Event]
def clear() -> None
```

File: nyawa/system.py

```
class SystemManager:
def __init__(self, event_bus: EventBus)
def init_gpio() -> None # Setup semua GPIO pin
def register_shutdown() -> None # SIGTERM + SIGINT handler
def graceful_shutdown() -> None # Urutan: mulut→wajah→telinga→GPIO cleanup
def reboot() -> None
# GPIO.setmode(GPIO.BCM)
# GPIO.setwarnings(False)
# Soft power button: GPIO.add_event_detect(3, GPIO.FALLING, ...)
```

4. Entry Point: main.py

```
#!/usr/bin/env python3
"""EMO Robot — Main Entry Point"""
import signal, logging
from config import Config
from nyawa.event_bus import EventBus
from nyawa.system import SystemManager
from nyawa.logger import setup_logger
from wajah.display import DisplayDriver
from wajah.animator import Animator
from jiwa.mood import MoodEngine
from jiwa.events import EventHandler
from telinga.touch import TouchSensor
from telinga.proximity import ProximitySensor
from telinga.microphone import MicrophoneListener
from mulut.speaker import Speaker
from mulut.tts import TextToSpeech
def main():
cfg = Config()
setup_logger(cfg.LOG_LEVEL)
bus = EventBus()
# Init semua modul
system = SystemManager(bus)
display = DisplayDriver(cfg.display)
speaker = Speaker(bus, cfg.audio)
tts = TextToSpeech(bus, cfg.audio)
mood = MoodEngine(bus)
anim = Animator(display, mood)
touch = TouchSensor(bus)
prox = ProximitySensor(bus, cfg.mic)
mic = MicrophoneListener(bus, cfg.mic)
handler = EventHandler(bus, mood, speaker, tts, anim)
# Start semua
system.init_gpio()
display.init()
speaker.init()
touch.start()
prox.start()
mic.start()
anim.start() # 20 FPS animation loop
handler.start() # Event processing loop
speaker.play('startup')
logging.info('EMO Robot ready!')
system.register_shutdown()
handler.join() # Block sampai shutdown
if __name__ == '__main__':
main()
```

5. Dependencies & Setup

requirements.txt

```
# Display
st7789==0.0.4
Pillow>=10.0.0
# GPIO
RPi.GPIO>=0.7.1
gpiozero>=2.0
# Audio
pygame>=2.5.0
pyaudio>=0.2.13
# Vision (Phase 2)
opencv-python-headless>=4.8.0
# face_recognition>=1.3.0 # uncomment Phase 2
# picamera2>=0.3.0 # uncomment Phase 2
# AI (Phase 3)
# openai>=1.0.0 # uncomment Phase 3
# Utils
numpy>=1.24.0
dataclasses-json>=0.6.0
```

setup.sh — Script install otomatis

```
#!/bin/bash
set -e
echo '=== EMO Robot Setup ==='
# System dependencies
sudo apt update
sudo apt install -y python3-pip python3-venv git i2c-tools \
espeak-ng alsa-utils portaudio19-dev
# Enable SPI
sudo raspi-config nonint do_spi 0
# Create venv
python3 -m venv venv
source venv/bin/activate
pip install --upgrade pip
pip install -r requirements.txt
# Create log dir
sudo mkdir -p /var/log/emorobot
sudo chown pi:pi /var/log/emorobot
# Install systemd service
sudo cp emorobot.service /etc/systemd/system/
sudo systemctl daemon-reload
sudo systemctl enable emorobot
echo '=== Setup selesai! Reboot untuk autostart ==='
```

emorobot.service — Systemd autostart

```
[Unit]
Description=EMO Robot Service
After=network.target sound.target
[Service]
Type=simple
User=pi
WorkingDirectory=/home/pi/emorobot
ExecStart=/home/pi/emorobot/venv/bin/python main.py
Restart=on-failure
RestartSec=5
StandardOutput=append:/var/log/emorobot/robot.log
StandardError=append:/var/log/emorobot/error.log
[Install]
WantedBy=multi-user.target
```

6. Instruksi untuk Claude Code Agent

Baca bagian ini dengan seksama. Ini adalah instruksi langsung untuk Claude Code Agent.

1	**Buat seluruh struktur folder** Buat semua folder: wajah/, jiwa/, telinga/, mulut/, mata/, otak/, nyawa/ beserta __init__.py di masing-masing folder.
2	**Implementasi Phase 1 penuh** Implementasi lengkap untuk modul: wajah, jiwa, telinga (touch + prox saja), mulut, nyawa. Modul mata dan otak cukup stub.
3	**Generate placeholder sound files** Buat script generate_sounds.py yang menggunakan espeak-ng untuk generate semua .wav placeholder di mulut/assets/sounds/.
4	**Buat config.py terpusat** Semua konfigurasi dalam 1 file config.py dengan dataclass per modul. Tidak ada hardcode di file modul manapun.
5	**Test setiap modul** Buat tests/ folder dengan test_wajah.py, test_jiwa.py, test_mulut.py. Minimal 3 test case per modul. Gunakan unittest.mock untuk GPIO.
6	**Setup.sh harus idempotent** Jalankan berulang kali harus aman — cek dulu sebelum install.
7	**README.md lengkap** Dokumentasi: cara install, cara run, cara test, cara shutdown, troubleshooting umum.
8	**Jangan install package yang tidak ada di requirements.txt** Kalau butuh package baru, tambahkan ke requirements.txt dan jelaskan alasannya.

Prompt yang disarankan untuk Claude Code: "Baca file PRD_ClaudeCode_EMORobot.pdf ini dan implementasikan seluruh struktur folder dan kode sesuai spesifikasi. Mulai dari struktur folder, lalu config.py, lalu nyawa/, lalu wajah/, jiwa/, mulut/, telinga/. Buat semua file termasuk __init__.py, tests/, setup.sh, emorobot.service, dan README.md."